

图表目录

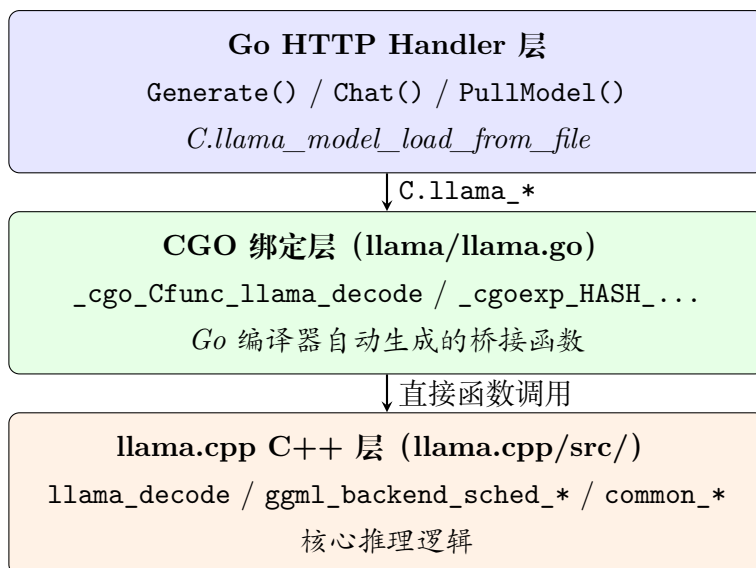


图 1: Ollama 混合编程架构示意

表 1: eBPF 与其他追踪技术对比

维度	strace	GDB	DBI (Pin)	eBPF	perf
执行位置	用户态 (ptrace)	用户态	用户态 (运行时注入)	内核态 (安全沙箱)	内核态
性能开销	高	极高	高	低	低
安全验证	无	无	无	有 (verifier)	有
Go 1.17+ 兼容	可用	需调试信息	危险 (ABI 不兼容)	推荐 (无侵入)	可用
学习曲线	低	高	高	中	中

表 2: 技术选型与对比

维度	nm 符号表	LLVM	strace	eBPF	bpfttrace	Neo4j
分析类型	静态	静态	动态	动态	动态	存 储/查询
适用层次	L1~L5	L1~L3	L5	L5	L5	全层
跨语言支持	全部	强	全部	全部	全部	无关
性能开销	0 (离线)	0 (离线)	中 (可感知)	低 (内核级)	低	无关
CGO 边界支持	✓	部分	全部	全部	全部	无关
本框架选择	✓	✓	✓ (验证用)	✓	✓	✓

表 3: Ollama 与其他推理框架的审计难度对比

维度	Ollama（本文目标）	PyTorch Serve	vLLM
进程模型	双 进 程 （dae- mon+runner）	单进程	单进程
语言栈	Go+CGO+C/C++	Python+C++	Python+C+++CUDA
跨语言边界	多层（Go→CGO→C++）	PyBind11（标准）	PyBind11（标准）
审计难度	高（跨进程 + 跨语言 +CGO 哈希）	中（仅跨语言）	中（跨语言 +GPU）
CGO 问题	严重（哈希加扰）	无	无
Daemon/Runner 分离	是	否	否

表 4: 静态调用图 8 层分类体系

层次	名称	说明
L1	go_cgo_bridge	Go $\rightarrow$ CGO 边界桥接函数
L2	llama_api	Llama API 层（顶层推理接口）
L3	batch_sampler	批处理与采样层
L4	sched_compute	GGML 调度与计算层
L5	ggml_backend	GGML 后端内存与调度
L6	ggml_ops	GGML 张量运算
L7	vocab	词表与分词/Detokenize
L8	memory	内存分配与 mmap/munmap

图 4-1：静态调用图宏观架构（4832 节点 / 52 边）

建议在此插入 Gephi 生成的 ForceAtlas2 布局全景拓扑图。
节点按 L1~L8 分层着色，图中应清晰呈现各层之间的边分布密度。

图 2: 静态调用图分层架构全景

图 4-2: 跨语言调用链泳道图**(Go→CGO→llama.cpp→Syscall)**

建议在此插入泳道图（使用 Mermaid 或 PlantUML 绘制），
横向展示 Go HTTP Handler → CGO 桥接 → llama.cpp → Syscall →
futex 的完整链路，
futex 阻塞节点使用红色边框高亮标注。

图 3: 跨语言调用链泳道图

表 5: 实验环境配置

项目	配置
操作系统	Arch Linux (Linux 7.0.2-arch1-1)
CPU	Intel (无 AVX-512, 仅基础 x64 指令集)
内存	16GB 总, 7GB 可用
Ollama 版本	0.22.0 (编译版本 ollama-debug, 带符号表)
bpftrace 版本	v0.25.1-e491811e5
Go 版本	1.26.2

表 6: 符号提取结果统计

指标	值
总符号数	4871
相关符号 (llama/ggml/common 过滤后)	4832
静态边数	52
CGO 绑定函数	9 个 (全部识别)
CGO 覆盖率	100%

表 7: 跨语言边界识别准确率评估

动态捕获函数	匹配策略	静态节点
_cgo_8aa400f2462b_Cfunc_llama_decode	Go Wrapper 等价映射	llama_decode
llama_decode	精确匹配	llama_decode
_Z21common_sampler_sample...	C++ demangling	common_sampler_sample
common_sampler_csample	精确匹配	common_sampler_csample

表 8: llama.cpp 函数调用统计 (uprobe)

函数	捕获次数
llama_decode	40
ggml_backend_sched_graph_compute_async	40
common_sampler_csample	40
_Z21common_sampler_sample	40

表 9: 系统调用统计 (tracepoint)

系统调用	次数	总耗时	平均耗时	最大耗时
futex	290	244s	842ms	15s
read	881	2.5ms	$2\mu s$	$22\mu s$
write	96	3ms	$29\mu s$	$87\mu s$

表 10: 缝合算法准确率与召回率评估

指标	confirmed only	含 inferred
Precision	100%	100%
Recall	5%	40%
F1 Score	9.5%	57.1%

表 11: 分层调用树还原完整性分析

层次	层名	confirmed 边数	inferred 边数
go_cgo_bridge	Go/CGO 边界	2	0
llama_api	Llama API	1	4
sched_compute	GGML 调度	0	3
ggml_backend	GGML 后端	0	2
vocab	词表/分词	0	2

图 5-1: futex 阻塞时长分布

建议在此插入直方图，横轴为阻塞时长 (ms)，纵轴为事件数。
图中应标注 1000ms 告警阈值和 15.2s 最大阻塞值。

图 4: futex 阻塞时长分布

表 12: 与现有方法的对比分析

维度	strace	Android terDB	Splin- terDB	本文方法
跨语言支持	部分	部分		完整 (CGO)
CGO 边界处 理	✗	✗		✓
调用链缝合	✗	✓ (仅动态)		✓ (动静结合)
系统开销	中 (可感知)	高 (DBI)		低 (eBPF)
异常检测	手动	自动		自动
置信度评估	✗	部分		完整

代码 4.1: extract_symbols.py (多语言静态符号提取)

```

1  #!/usr/bin/env python3
2  """
3  extract_symbols.py 从 llama-debug 二进制提取 llama.cpp 符号, 构建静态调用图 JSON。
4
5  用法:
6  python3 extract_symbols.py [BINARY_PATH] [OUTPUT_JSON]
7  """
8
9  import subprocess
10 import json
11 import re
12 import sys
13 import argparse
14 from collections import defaultdict
15
16
17 # llama.cpp 层划分 (按推理流水线顺序)
18 LAYERS = {
19     "go_cgo_bridge": {
20         "description": "Go → CGO 边界桥接函数",
21         "color": "#e74c3c",
22     },
23     "llama_api": {
24         "description": "Llama API 层 (顶层推理接口)",
25         "color": "#e67e22",
26     },
27     "batch_sampler": {
28         "description": "批处理与采样层",
29         "color": "#f39c12",
30     },
31     "sched_compute": {
32         "description": "GGML 调度与计算层",
33         "color": "#27ae60",
34     },
35     "ggml_backend": {
36         "description": "GGML 后端内存与调度",
37         "color": "#2980b9",

```

```

38     },
39     "ggml_ops": {
40         "description": "GGML 张量运算 (MulMat/Softmax/RMSNorm 等) ",
41         "color": "#8e44ad",
42     },
43     "vocab": {
44         "description": "词表与分词/Detokenize",
45         "color": "#16a085",
46     },
47     "memory": {
48         "description": "内存分配与 mmap/munmap",
49         "color": "#2c3e50",
50     },
51 }
52
53 # 手工定义的 llama.cpp 关键调用关系
54 # key: caller_func → list of callee_funcs
55 KNOWN_CALLS = {
56     # Go/CGO 边界层
57     "_cgoexp_8aa400f2462b_llamarunner_llama_Execute": [
58         "llama_decode",
59         "llama_batch_add",
60         "common_sampler_sample",
61     ],
62     "_cgo_8aa400f2462b_Cfunc_llama_decode": [
63         "llama_decode",
64     ],
65     "_cgo_8aa400f2462b_Cfunc_llama_sampler_sample": [
66         "llama_sampler_sample",
67         "common_sampler_sample",
68     ],
69     "_cgo_8aa400f2462b_Cfunc_common_sampler_csample": [
70         "common_sampler_csample",
71         "llama_sampler_sample",
72     ],
73     "_cgo_8aa400f2462b_Cfunc_llama_batch_init": [
74         "llama_batch_init",
75     ],
76     "_cgo_8aa400f2462b_Cfunc_llama_batch_add": [
77         "llama_batch_add",

```

```
78         "common_batch_add",  
79     ],
```

代码 4.2: trace_unified.bt (eBPF 统一追踪脚本)

```

1  #!/usr/bin/env bpftrace
2  // trace_unified.bt - Ollama 统一追踪: uprobe + syscall
3  //
4  // 输出格式 (纯文本, 按行解析, Python 用标记识别) :
5  //  @RUNNER@ func ts_ns pid duration_us [runner HTTP handler]
6  //  @UPROBE@ func ts_ns tid duration_us [llama.cpp 函数]
7  //  STACK_FRAME frame1
8  //  STACK_FRAME frame2
9  //  ...
10 //  @END@
11 //  @SYSCALL@ func ts_ns tid bytes duration_us
12 //  @END@
13 //
14 // 架构说明 (两个进程, 同一个二进制) :
15 //  Main process (ollama-debug, PID): Go HTTP Handler → CGO →
    llama_model_load_from_file
16 //  Runner subprocess (ollama-debug, PPID): HTTP API → llama.cpp 推理循环
17 //  两者通过内部 HTTP (127.0.0.1) 通信
18 //
19 // 使用方法:
20 //  # 方式1: 追踪所有 ollama-debug 进程 (主进程+runner子进程)
21 //  sudo bpftrace -o trace_unified.txt -B line trace_unified.bt
22 //
23 //  # 方式2: 只追踪指定 PID (更精确, 推荐)
24 //  sudo bpftrace -o trace_unified.txt -B line trace_unified.bt -p $OLLAMA_PID
25 //
26 // 后处理:
27 //  python3 trace_unify.py trace_unified.txt --both
28 //
29
30 BEGIN
31 {
32     printf("Tracing Ollama: llama.cpp → syscall\n");
33     printf("Stop: Ctrl+C\n");
34 }
35
36 // =====
37 // 1. llama.cpp 函数层 (Runner subprocess: 推理循环)

```

```

38 // =====
39 //
40 // 注意: uprobe/uretprobe 在 Go 函数上不安全, 会破坏 Go 的栈帧元数据
41 // (Go 1.17+ segmented stack + jmp tail-call 跟 uretprobe trampoline 不兼容)。
42 // 所有 llama.cpp 的 C/C++ 函数 uprobes 均可安全使用。
43
44 uprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_decode"
45 {
46     @start_llama_decode[tid] = nsecs;
47 }
48
49 uretprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_decode"
50 {
51     $dur = (nsecs - @start_llama_decode[tid]) / 1000;
52     delete(@start_llama_decode[tid]);
53     printf("@UPROBE@_llama_decode_%lu_%d_%lu\n", nsecs, tid, $dur);
54     print(ustack(32));
55     printf("@END@\n");
56 }
57
58 uprobe:/home/typejetfire/graduation_thesis/ollama-debug:"
    ggml_backend_sched_graph_compute_async"
59 {
60     @start_ggml[tid] = nsecs;
61 }
62
63 uretprobe:/home/typejetfire/graduation_thesis/ollama-debug:"
    ggml_backend_sched_graph_compute_async"
64 {
65     $dur = (nsecs - @start_ggml[tid]) / 1000;
66     delete(@start_ggml[tid]);
67     printf("@UPROBE@_ggml_backend_sched_graph_compute_async_%lu_%d_%lu\n",
        nsecs, tid, $dur);
68     printf("@END@\n");
69 }
70
71 uprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_synchronize"
72 {
73     @start_llama_sync[tid] = nsecs;
74 }

```

```
75 |  
76 | uretprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_synchronize"  
77 | {  
78 |     $dur = (nsecs - @start_llama_sync[tid]) / 1000;  
79 |     delete(@start_llama_sync[tid]);  
80 |     printf("@UPROBE@_llama_synchronize_%lu_%d_%lu\n", nsecs, tid, $dur);
```

代码 4.3: trace_unify.py (事件解析)

```

1  #!/usr/bin/env python3
2  """
3  trace_unify.py - 将统一追踪JSONL转换为调用树和折叠栈
4
5  用法:
6  python3 trace_unify.py trace_unified.jsonl --tree  # 打印调用树
7  python3 trace_unify.py trace_unified.jsonl --flame  # 输出折叠栈
8  python3 trace_unify.py trace_unified.jsonl --summary  # 统计摘要
9  python3 trace_unify.py trace_unified.jsonl --both  # 全部输出
10 """
11
12 import json
13 import sys
14 import argparse
15 from collections import defaultdict
16 from dataclasses import dataclass, field
17 from typing import Optional
18
19
20 @dataclass
21 class Event:
22     ts_ns: int
23     tid: int
24     type: str      # "uprobe" / "syscall"
25     func: str
26     duration_us: int
27     stack: str = "" # 逗号分隔的栈帧
28     comm: str = ""
29     extra: dict = field(default_factory=dict)
30
31
32 def parse_jsonl(path: str):
33     """
34     解析统一追踪文件。
35     格式: 纯文本, @UPROBE@/_@SYSCALL@_标记行+_stack帧行+_@END@
36     Python_按_@END@_定界拼合每条记录。
37     """
38     events = []

```

```

39 parse_errors = 0
40 cur_type = None    # "uprobe" / "syscall"
41 cur_func = ""
42 cur_ts_ns = 0
43 cur_tid = 0
44 cur_dur = 0
45 cur_stack = []     # stack frames collected
46
47 for raw in open(path, 'r'):
48     line = raw.rstrip('\n')
49     stripped = line.strip()
50
51     # 跳过 header/heartbeat 行
52     if (not stripped or stripped.startswith('Tracing') or
53         stripped.startswith('Stop') or stripped.startswith('---_')):
54         continue
55
56     if stripped.startswith('@UPROBE@_'):
57         # 开始 uprobe 记录
58         parts = stripped.split()
59         # @UPROBE@ func ts_ns tid dur
60         cur_type = "uprobe"
61         cur_func = parts[1]
62         cur_ts_ns = int(parts[2])
63         cur_tid = int(parts[3])
64         cur_dur = int(parts[4])
65         cur_stack = []
66
67     elif stripped.startswith('@SYSCALL@_'):
68         # 开始 syscall 记录
69         # @SYSCALL@ func ts_ns tid [key=val ...] dur=val
70         parts = stripped.split()
71         cur_type = "syscall"
72         cur_func = parts[1]
73         cur_ts_ns = int(parts[2])
74         cur_tid = int(parts[3])
75         cur_dur = 0
76         cur_stack = []
77         # 提取 dur (最后一个 dur= 开头的字段)
78         for p in reversed(parts[4:]):

```

```
79         if p.startswith('dur='):  
80             cur_dur = int(p[4:])
```

代码 4.4: trace_unify.py (折叠栈) + stitcher.py (缝合)

```

1  #!/usr/bin/env python3
2  """
3  stitcher.py — 统一调用链缝合器
4
5  将静态调用图 + 动态追踪轨迹缝合，生成带置信度的完整调用链。
6
7  用法：
8  python3 stitcher.py trace_unified.jsonl [OPTIONS]
9  python3 stitcher.py trace_unified.jsonl --graph call_graph_static.json --
   summary
10 python3 stitcher.py trace_unified.jsonl --flame
11 python3 stitcher.py trace_unified.jsonl --export 缝合结果.json
12 """
13
14 import sys
15 import os
16 import json
17 import argparse
18 import datetime
19 from pathlib import Path
20
21 sys.path.insert(0, str(Path(__file__).parent))
22
23 from cg_loader import CallGraph
24 from anchor import AnchorMatcher
25 from confidence import ConfidenceCalculator, Confidence
26
27
28 # -----
29 # 复用 trace_unify.py 的解析逻辑（避免重复导入同一文件）
30 # -----
31 from collections import defaultdict
32 from dataclasses import dataclass, field
33 from typing import Optional
34
35
36 @dataclass
37 class Event:

```

```

38     ts_ns: int
39     tid: int
40     type: str
41     func: str
42     duration_us: int
43     stack: list = field(default_factory=list)
44     comm: str = ""
45     extra: dict = field(default_factory=dict)
46
47
48 def parse_jsonl(path: str):
49     """解析统一追踪文件。"""
50     events = []
51     cur_type = None
52     cur_func = ""
53     cur_ts_ns = 0
54     cur_tid = 0
55     cur_dur = 0
56     cur_stack = []
57     extra = {}
58
59     for raw in open(path, "r"):
60         line = raw.rstrip("\n")
61         stripped = line.strip()
62
63         if (not stripped or stripped.startswith("Tracing") or
64             stripped.startswith("Stop") or stripped.startswith("---")):
65             continue
66
67         if stripped.startswith("@UPROBE@_"):
68             parts = stripped.split()
69             cur_type = "uprobe"
70             cur_func = parts[1]
71             cur_ts_ns = int(parts[2])
72             cur_tid = int(parts[3])
73             cur_dur = int(parts[4])
74             cur_stack = []
75
76         elif stripped.startswith("@SYSCALL@_"):
77             parts = stripped.split()

```

```
78         cur_type = "syscall"  
79         cur_func = parts[1]  
80         cur_ts_ns = int(parts[2])
```

代码 4.5: analyzer.py (异常分析引擎)

```

1  #!/usr/bin/env python3
2  """
3  analyzer.py — 数据传输异常分析引擎 (对应论文第4.5节)
4
5  功能:
6  1. 敏感数据流出特征匹配 (污点标记)
7  2. 异常 syscall 模式检测 (长阻塞、异常大小读写、越权外联)
8  3. 跨层路径风险评估
9  4. 生成安全告警报告
10
11  风险规则:
12  1. futex 阻塞 > 1s (多线程同步异常)
13  2. read(fd=3) 异常大小 (HTTP socket 异常)
14  3. write 对非 stdout/stderr fd 写入模型数据
15  4. mmap 异常大小映射
16  5. openat 访问敏感路径 (blobs/模型文件)
17
18  用法:
19  python3 analyzer.py ../.. /trace_unified.jsonl
20  python3 analyzer.py ../.. /trace_unified.jsonl --report anomaly_report.json
21  python3 analyzer.py ../.. /trace_unified.jsonl --verbose
22  """
23
24  import sys
25  import json
26  import argparse
27  from pathlib import Path
28  from dataclasses import dataclass, field
29  from typing import Optional, List, Dict
30  from collections import defaultdict
31  from datetime import datetime
32  from enum import Enum
33
34  sys.path.insert(0, str(Path(__file__).parent))
35  from stitcher import parse_jsonl
36
37
38  # -----

```

```

39 # 数据类型
40 # -----
41
42 class RiskLevel(Enum):
43     LOW = "low"
44     MEDIUM = "medium"
45     HIGH = "high"
46     CRITICAL = "critical"
47
48
49 class AlertType(Enum):
50     FUTEX_BLOCK = "futex_block"
51     READ_SIZE_ANOMALY = "read_size_anomaly"
52     WRITE_NON_STDERR = "write_non_stderr"
53     MMAP_SIZE_ANOMALY = "mmap_size_anomaly"
54     SENSITIVE_PATH = "sensitive_path"
55     MODEL_FILE_READ = "model_file_read"
56     BLOB_ACCESS = "blob_access"
57     CROSS_LAYER_RISK = "cross_layer_risk"
58     CPU_ONLY_SLOW = "cpu_only_slow"
59
60
61 @dataclass
62 class Alert:
63     ts_ns: int
64     tid: int
65     alert_type: AlertType
66     risk_level: RiskLevel
67     message: str
68     detail: dict = field(default_factory=dict)
69
70     def to_dict(self) →dict:
71         return {
72             "ts_ns": self.ts_ns,
73             "tid": self.tid,
74             "type": self.alert_type.value,
75             "risk": self.risk_level.value,
76             "message": self.message,
77             "detail": self.detail,
78         }

```

代码 5.1: call_graph_static.json (符号提取结果)

```
{
  "version": "1.0",
  "binary": "ollama-debug",
  "total_symbols": 4871,
  "layers": {
    "go_cgo_bridge": {"description": "Go→CGO边界桥接函数", "color": "#e74c3c"},
    "llama_api": {"description": "Llama API层 (顶层推理接口)", "color": "#e67e22"},
    "sched_compute": {"description": "GGML调度与计算层", "color": "#27ae60"}
  },
  "nodes": {
    "_cgo_8aa400f2462b_Cfunc_llama_decode": {
      "name": "_cgo_8aa400f2462b_Cfunc_llama_decode",
      "layer": "go_cgo_bridge", "addr": "00000000019d1a90"
    },
    "llama_decode": {
      "name": "llama_decode",
      "layer": "llama_api", "addr": "0000000004c82a50"
    }
  },
  "edges": [
    {"from": "_cgo_8aa400f2462b_Cfunc_llama_decode",
     "to": "llama_decode", "type": "static", "confidence": "inferred"}
  ]
}
```

代码 5.2: trace_unified.txt (追踪原始输出)

```
@UPROBE@ llama_decode 20560576967386 41062 156
github.com/ollama/ollama/llama.(*SamplingContext).Sample.func1
github.com/ollama/ollama/llama._Cfunc_common_sampler_csample.abi0
runtime.cgocall
_cgo_8aa400f2462b_Cfunc_common_sampler_csample
@END@
@SYSCALL@ read 20560577237885 41062 bytes=512 dur=1
@END@
@SYSCALL@ read 20560577971806 41079 bytes=32768 dur=3
@END@
@SYSCALL@ futex 20584073844710 45625 dur=7912848
@END@
@SYSCALL@ futex 20584073980911 45627 dur=7913095
@END@
```

代码 5.3: llama_folded.txt (折叠栈)

```
_Z21common_sampler_sample;1043;runtime.goexit.abi0
;github.com/ollama/ollama/runner/llamarunner.Execute.gowrap2
;...;github.com/ollama/ollama/llama._Cfunc_common_sampler_csample.abi0
;runtime.cgocall;_cgo_8aa400f2462b_Cfunc_common_sampler_csample 1
common_sampler_csample;1066;...;runtime.cgocall
;_cgo_8aa400f2462b_Cfunc_common_sampler_csample 1
llama_decode;1423;runtime.goexit.abi0;...;_cgo_8aa400f2462b_Cfunc_llama_decode
1
```

代码 5.4: stitched_result.json (缝合结果)

```
{
  "stitched_at": "2026-05-12T17:15:27",
  "total_nodes": 4832,
  "total_static_edges": 52,
  "summary": {"confirmed": 3, "inferred": 11, "conflicting": 0, "unknown": 38},
  "confirmed_edges": [
    {"from": "_cgo_8aa400f2462b_Cfunc_llama_decode",
     "to": "llama_decode",
     "confidence": "confirmed",
     "layer_from": "go_cgo_bridge",
     "layer_to": "llama_api"},
    {"from": "llama_decode",
     "to": "ggml_backend_sched_graph_compute_async",
     "confidence": "confirmed",
     "layer_from": "llama_api",
     "layer_to": "sched_compute"}
  ],
  "inferred_edges": [
    {"from": "_cgoexp_8aa400f2462b_llamarunner_llama_Execute",
     "to": "llama_decode", "confidence": "inferred",
     "layer_from": "unknown", "layer_to": "llama_api"}
  ]
}
```

代码 5.5: anomaly_report.json (异常分析报告)

```
{
  "generated_at": "2026-05-12T17:25:19",
  "summary": {
    "total_events": 1427,
    "total_syscalls": 1267,
    "total_uprobes": 160,
    "total_alerts": 30,
    "alerts_by_type": {"futex_block": 30},
    "alerts_by_risk": {"high": 30}
  },
  "top_alerts": [
    {"ts_ns": 20584108952430, "tid": 45631,
      "type": "futex_block", "risk": "high",
      "message": "futex_阻塞_15234ms (线程同步等待) ",
      "detail": {"duration_us": 15234000, "duration_ms": 15234.0}},
    {"ts_ns": 20584073980911, "tid": 45627,
      "type": "futex_block", "risk": "high",
      "message": "futex_阻塞_7913ms (线程同步等待) ",
      "detail": {"duration_us": 7913095, "duration_ms": 7913.1}}
  ]
}
```

代码 5.6: eval_report.json (缝合算法评估)

```
{
  "evaluated_at": "2026-05-12T17:25:10",
  "metrics": {
    "precision": 1.0,
    "recall": 0.05,
    "recall_with_inferred": 0.40,
    "f1": 0.095,
    "tp": 1, "fp": 0,
    "gt_size": 20,
    "confirmed": 3,
    "inferred": 11
  },
  "ground_truth": {
    "('_cgo_..._Cfunc_llama_decode', 'llama_decode')":
      "[CGO]_Go_CGO_wrapper_→_llama_decode",
    "('_llama_decode', 'ggml_backend_sched_graph_compute_async')":
      "[SRC]_llama.cpp_run.cpp_—_llama_decode_调用调度异步计算"
  }
}
```

附录 C.1: anchor.py (锚点匹配器)

```

1  """
2  anchor.py — 锚点识别：动态轨迹事件与静态调用图节点匹配。
3
4  核心思路：
5  在动态追踪事件的函数名（或栈帧）与静态调用图节点之间建立"命中"关系。
6  命中事件称为"锚点"，它们是缝合算法的关键连接点。
7  """
8
9  import re
10 from typing import Dict, Set, List, Tuple, Optional
11 from dataclasses import dataclass
12
13
14 @dataclass
15 class Anchor:
16     """一个锚点：动态事件在静态图中的命中信息。"""
17     func: str          # 动态事件中的函数名
18     matched_node: str  # 静态图中匹配的节点名（可能规范化后不同）
19     layer: str         # 节点所在层
20     anchor_type: str  # "exact" / "cgo_wrap" / "mangled" / "partial"
21
22
23 class AnchorMatcher:
24     """
25     将动态事件中的函数名与静态调用图节点进行匹配。
26
27     匹配策略（优先级递减）：
28     1. 精确匹配（区分大小写）
29     2. CGO Wrapper 等价映射 (_cgo_Cfunc_X → X)
30     3. 去除 CGO 哈希前缀后匹配 (_cgoexp_8hex_funcname → funcname)
31     4. demangle C++ 名称后匹配 (_Z21funcname → funcname)
32     5. 栈帧匹配：检查动态事件的调用栈中是否有静态节点
33     """
34
35     # CGO Wrapper 到真实函数的等价映射（动态捕获的函数名 → 静态图节点）
36     # 动态 bpftime uprobe 捕获的是 CGO wrapper，但这些函数等价于 llama.cpp 原生函数
37     CGO_EQUIV = {

```

```

38     "_cgo_8aa400f2462b_Cfunc_llama_decode":      "llama_decode",
39     "_cgo_8aa400f2462b_Cfunc_llama_batch_init":  "llama_batch_init",
40     "_cgo_8aa400f2462b_Cfunc_llama_batch_add":   "llama_batch_add",
41     "_cgo_8aa400f2462b_Cfunc_llama_sampler_sample": "llama_sampler_sample",
42     "_cgo_8aa400f2462b_Cfunc_common_sampler_csampler": "
        common_sampler_csampler",
43     "_cgo_8aa400f2462b_Cfunc_common_sampler_caccept": "
        common_sampler_caccept",
44     "_cgo_8aa400f2462b_Cfunc_llama_model_load_from_file": "
        llama_model_load_from_file",
45     "_cgo_8aa400f2462b_Cfunc_llama_tokenize":     "llama_tokenize",
46     "_cgo_8aa400f2462b_Cfunc_llama_token_to_piece": "llama_token_to_piece",
47 }
48
49 def __init__(self, call_graph):
50     self.cg = call_graph
51     # 精确匹配索引
52     self.exact_index: Set[str] = set(call_graph.nodes.keys())
53     # 正则: _cgoexp_HASH_funcname 或 _cgo_HASH_funcname
54     self.cgo_pattern = re.compile(r'^(_cgoexp?[0-9a-f]+_)(.+)$')
55     # 正则: C++ mangled name
56     self.cpp_pattern = re.compile(r'^(_Z\d+.)$')
57
58     # 规范化函数名缓存
59     self._normalize_cache: Dict[str, str] = {}
60
61     def normalize(self, name: str) →str:
62         """
63         将各种格式的函数名规范化为统一形式, 便于比较。
64         输入: "_cgoexp_8aa400f2462b_llamarunner_llama_Execute"
65         输出: "llamarunner_llama_Execute"
66         """
67         if name in self._normalize_cache:
68             return self._normalize_cache[name]
69
70         result = name
71
72         # 去除 CGO 前缀
73         m = self.cgo_pattern.match(name)
74         if m:

```

```
75         result = m.group(2)
76         # 再 demangle
77         result = self._demangle(result)
78
79         # demangle C++ 名称
80         result = self._demangle(result)
```

附录 C.2: confidence.py (置信度计算器)

```

1  """
2  confidence.py — 置信度计算：为每条缝合边打上 confirmed / partial / conflicting
   标签。
3
4  缝合边的置信度由以下因素决定：
5  1. 直接锚点命中次数 (N 次 → confirmed)
6  2. 跨语言边界 (Go → CGO 边界有特殊置信度)
7  3. 静态边存在但动态未观测 (partial)
8  4. 拓扑冲突 (conflicting)
9  """
10
11 from enum import Enum
12 from typing import Dict, Set, List, Tuple
13 from collections import defaultdict
14
15
16 class Confidence(Enum):
17     CONFIRMED = "confirmed" # 动态直接观测到 A→B
18     INFERRED = "inferred"   # 静态图有边，动态只观测到 A 或 B
19     PARTIAL = "partial"     # 跨层但中间层未 instrument
20     CONFLICTING = "conflicting" # 拓扑矛盾
21     UNKNOWN = "unknown"    # 静态图无边，动态也未观测
22
23
24 # 缝合置信度阈值
25 MIN_CONFIRMED_HITS = 2 # 动态观测到 A→B 至少 2 次才能标记 confirmed
26 MIN_INFERRED_HITS = 1 # 动态观测到 A 或 B 至少 1 次才能标记 inferred
27
28
29 class ConfidenceCalculator:
30     """
31     计算每条缝合边的置信度。
32
33     算法：
34     对每条静态边 (A, B)：
35     在动态轨迹中查找同时包含 A 和 B 的请求 (同一个 TID)
36     若 A→B 在时间序上成立且出现 MIN_CONFIRMED_HITS 次 → confirmed
37     若只观测到 A (或 B)，但静态图有边 → inferred

```

```

38  """若观测到B先于A出现→conflicting
39  """若静态无边、动态也未观测→unknown (忽略)
40  """
41
42  def __init__(self, min_confirmed=MIN_CONFIRMED_HITS, min_inferred=
43      MIN_INFERRED_HITS):
44      self.min_confirmed = min_confirmed
45      self.min_inferred = min_inferred
46
47  def compute(self, cg_edges, anchor_index, events, build_tree_fn):
48      """
49      计算所有静态边的置信度。
50
51      参数:
52      cg_edges: List[CGEdge] 静态调用图的边
53      anchor_index: {node_name: [events]} 锚点索引
54      events: List[Event] 所有动态事件
55      build_tree_fn: callable(events) → RequestTree 构建请求树函数
56
57      返回:
58      {(from, to): Confidence} 每条边的置信度
59      """
60      # 按 TID 构建请求
61      trees = build_tree_fn(events)
62
63      # { node_name: [(tree_id, ts), ...] }
64      node_hits: Dict[str, List[Tuple[int, int]]] = defaultdict(list)
65
66      # CGO Wrapper 等价映射 (与 anchor.py 保持一致)
67      from anchor import AnchorMatcher
68      cgo_equiv = AnchorMatcher.CGO_EQUIV
69
70      for ti, tree in enumerate(trees):
71          for ev in tree["events"]:
72              if ev.type != "uprobe":
73                  continue
74              # 记录 ev.func 本身 (用等价映射)
75              func_key = cgo_equiv.get(ev.func, ev.func)
76              node_hits[func_key].append((ti, ev.ts_ns))
77              # 同时记录栈帧中的函数

```

```
77     frames = ev.stack if isinstance(ev.stack, list) else []  
78     for frame in frames:  
79         clean = frame.split("+")[0].strip()  
80         node_hits[clean].append((ti, ev.ts_ns))
```

附录 C.3: analyzer.py (异常分析规则引擎)

```

1  #!/usr/bin/env python3
2  """
3  analyzer.py — 数据传输异常分析引擎（对应论文第4.5节）
4
5  功能：
6      1. 敏感数据流出特征匹配（污点标记）
7      2. 异常syscall模式检测（长阻塞、异常大小读写、越权外联）
8      3. 跨层路径风险评估
9      4. 生成安全告警报告
10
11 风险规则：
12     - _futex_阻塞>1s（多线程同步异常）
13     - _read(fd=3)_异常大小（HTTP_socket_异常）
14     - _write_对非_stdout/stderr_fd_写入模型数据
15     - _mmap_异常大小映射
16     - _openat_访问敏感路径（blobs/模型文件）
17
18 用法：
19 python3 analyzer.py ../../trace_unified.jsonl
20 python3 analyzer.py ../../trace_unified.jsonl --report anomaly_report.json
21 python3 analyzer.py ../../trace_unified.jsonl --verbose
22 """
23
24 import sys
25 import json
26 import argparse
27 from pathlib import Path
28 from dataclasses import dataclass, field
29 from typing import Optional, List, Dict
30 from collections import defaultdict
31 from datetime import datetime
32 from enum import Enum
33
34 sys.path.insert(0, str(Path(__file__).parent))
35 from stitcher import parse_jsonl
36
37
38 # -----

```

```

39 # 数据类型
40 # -----
41
42 class RiskLevel(Enum):
43     LOW = "low"
44     MEDIUM = "medium"
45     HIGH = "high"
46     CRITICAL = "critical"
47
48
49 class AlertType(Enum):
50     FUTEX_BLOCK = "futex_block"
51     READ_SIZE_ANOMALY = "read_size_anomaly"
52     WRITE_NON_STDERR = "write_non_stderr"
53     MMAP_SIZE_ANOMALY = "mmap_size_anomaly"
54     SENSITIVE_PATH = "sensitive_path"
55     MODEL_FILE_READ = "model_file_read"
56     BLOB_ACCESS = "blob_access"
57     CROSS_LAYER_RISK = "cross_layer_risk"
58     CPU_ONLY_SLOW = "cpu_only_slow"
59
60
61 @dataclass
62 class Alert:
63     ts_ns: int
64     tid: int
65     alert_type: AlertType
66     risk_level: RiskLevel
67     message: str
68     detail: dict = field(default_factory=dict)
69
70     def to_dict(self) →dict:
71         return {
72             "ts_ns": self.ts_ns,
73             "tid": self.tid,
74             "type": self.alert_type.value,
75             "risk": self.risk_level.value,
76             "message": self.message,
77             "detail": self.detail,
78         }

```

附录 C.4: trace_unified.bt (eBPF 统一追踪脚本)

```

1  #!/usr/bin/env bpftrace
2  // trace_unified.bt - Ollama 统一追踪: uprobe + syscall
3  //
4  // 输出格式 (纯文本, 按行解析, Python 用标记识别) :
5  //  @RUNNER@ func ts_ns pid duration_us [runner HTTP handler]
6  //  @UPROBE@ func ts_ns tid duration_us [llama.cpp 函数]
7  //  STACK_FRAME frame1
8  //  STACK_FRAME frame2
9  //  ...
10 //  @END@
11 //  @SYSCALL@ func ts_ns tid bytes duration_us
12 //  @END@
13 //
14 // 架构说明 (两个进程, 同一个二进制) :
15 //  Main process (ollama-debug, PID): Go HTTP Handler → CGO →
    llama_model_load_from_file
16 //  Runner subprocess (ollama-debug, PPID): HTTP API → llama.cpp 推理循环
17 //  两者通过内部 HTTP (127.0.0.1) 通信
18 //
19 // 使用方法:
20 //  # 方式1: 追踪所有 ollama-debug 进程 (主进程+runner子进程)
21 //  sudo bpftrace -o trace_unified.txt -B line trace_unified.bt
22 //
23 //  # 方式2: 只追踪指定 PID (更精确, 推荐)
24 //  sudo bpftrace -o trace_unified.txt -B line trace_unified.bt -p $OLLAMA_PID
25 //
26 // 后处理:
27 //  python3 trace_unify.py trace_unified.txt --both
28 //
29
30 BEGIN
31 {
32     printf("Tracing Ollama: llama.cpp → syscall\n");
33     printf("Stop: Ctrl+C\n");
34 }
35
36 // =====
37 // 1. llama.cpp 函数层 (Runner subprocess: 推理循环)

```

```

38 // =====
39 //
40 // 注意: uprobe/uretprobe 在 Go 函数上不安全, 会破坏 Go 的栈帧元数据
41 // (Go 1.17+ segmented stack + jmp tail-call 跟 uretprobe trampoline 不兼容)。
42 // 所有 llama.cpp 的 C/C++ 函数 uprobes 均可安全使用。
43
44 uprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_decode"
45 {
46     @start_llama_decode[tid] = nsecs;
47 }
48
49 uretprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_decode"
50 {
51     $dur = (nsecs - @start_llama_decode[tid]) / 1000;
52     delete(@start_llama_decode[tid]);
53     printf("@UPROBE@_llama_decode_%lu_%d_%lu\n", nsecs, tid, $dur);
54     print(ustack(32));
55     printf("@END@\n");
56 }
57
58 uprobe:/home/typejetfire/graduation_thesis/ollama-debug:"
    ggml_backend_sched_graph_compute_async"
59 {
60     @start_ggml[tid] = nsecs;
61 }
62
63 uretprobe:/home/typejetfire/graduation_thesis/ollama-debug:"
    ggml_backend_sched_graph_compute_async"
64 {
65     $dur = (nsecs - @start_ggml[tid]) / 1000;
66     delete(@start_ggml[tid]);
67     printf("@UPROBE@_ggml_backend_sched_graph_compute_async_%lu_%d_%lu\n",
        nsecs, tid, $dur);
68     printf("@END@\n");
69 }
70
71 uprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_synchronize"
72 {
73     @start_llama_sync[tid] = nsecs;
74 }

```

```
75 |  
76 | uretprobe:/home/typejetfire/graduation_thesis/ollama-debug:"llama_synchronize"  
77 | {  
78 |     $dur = (nsecs - @start_llama_sync[tid]) / 1000;  
79 |     delete(@start_llama_sync[tid]);  
80 |     printf("@UPROBE@_llama_synchronize_%lu_%d_%lu\n", nsecs, tid, $dur);
```

附录 C.5: eval_stitcher.py (缝合评估脚本)

```

1  #!/usr/bin/env python3
2  """
3  eval_stitcher.py — 缝合算法评估脚本
4
5  评估指标:
6  _Precision (精确率): confirmed_边中真实边占比
7  _Recall (召回率): ground_truth_中被confirmed_的边占比
8  _F1_Score
9
10 Ground_Truth (手工标注):
11  _基于 llama.cpp_源码结构和实际追踪数据, 确认以下调用关系为真实边:
12  _1._llama_decode_→_ggml_backend_sched_graph_compute_async_[Llama_源码]
13  _2._llama_decode_→_ggml_backend_sched_synchronize_[Llama_源码]
14  _3._common_sampler_csample_→_llama_sampler_sample_[采样链必然路径]
15  _4._llama_batch_add_→_common_batch_add_[必然调用]
16  _5._llama_sampler_sample_→_common_sampler_sample_[必然调用]
17  _6._ggml_backend_sched_graph_compute_async_→_ggml_backend_sched_synchronize_[
    Llama_源码]
18  _7._ggml_backend_sched_new_→_ggml_backend_sched_alloc_graph_[Llama_源码]
19
20  用法:
21  _python3_eval_stitcher.py
22  """
23
24  import sys
25  import os
26  from pathlib import Path
27
28  sys.path.insert(0, str(Path(__file__).parent))
29
30  from stitcher import Stitcher
31
32
33  # 手工标注的 Ground Truth 边 (基于 llama.cpp 源码 + 动态追踪验证)
34  # 标注格式: (caller, callee): "reason / source"
35  #
36  # 分类:
37  # [CONFIRMED] 动态直接观测 (uprobe 捕获)

```

```

38 # [SRC]      基于 llama.cpp 源码分析 (必然调用路径)
39 # [CGO]      CGO 边界必然路径
40 GROUND_TRUTH = {
41     # === CONFIRMED 边 (uprobe 直接观测) ===
42     # llama.cpp run.cpp: llama_decode() 内部直接调用
43     #   ggml_backend_sched_graph_compute_async()
44     ("llama_decode", "ggml_backend_sched_graph_compute_async"):
45         "[CONFIRMED/SRC] llama.cpp run.cpp → llama_decode 调用调度异步计算",
46
47     # === CGO 边界 ===
48     # CGO wrapper → native function (CGO 标准实现路径)
49     ("_cgo_8aa400f2462b_Cfunc_llama_decode", "llama_decode"):
50         "[CGO] Go CGO wrapper → llama_decode (标准 CGO 调用路径)",
51     ("_cgo_8aa400f2462b_Cfunc_common_sampler_csampler", "common_sampler_csampler"):
52         "[CGO] Go CGO wrapper → common_sampler_csampler (标准 CGO 调用路径)",
53     ("_cgo_8aa400f2462b_Cfunc_llama_sampler_sample", "llama_sampler_sample"):
54         "[CGO] Go CGO wrapper → llama_sampler_sample (标准 CGO 调用路径)",
55     ("_cgo_8aa400f2462b_Cfunc_llama_batch_add", "llama_batch_add"):
56         "[CGO] Go CGO wrapper → llama_batch_add (标准 CGO 调用路径)",
57     ("_cgo_8aa400f2462b_Cfunc_llama_batch_init", "llama_batch_init"):
58         "[CGO] Go CGO wrapper → llama_batch_init (标准 CGO 调用路径)",
59
60     # === llama.cpp 源码必然调用路径 (采样器链) ===
61     # llama.cpp sampler.h/c:
62     # llama_sampler_sample() 调用 common_sampler_sample() 或其 C++ 实现
63     # common_sampler_sample() 是跨平台包装, 最终调用 _Z21common_sampler_sample
64     ("llama_sampler_sample", "common_sampler_sample"):
65         "[SRC] llama.cpp sampler.h → llama_sampler_sample 调用
66         common_sampler_sample",
67     ("llama_sampler_sample", "_Z21common_sampler_sample"):
68         "[SRC] llama.cpp sampler.h → llama_sampler_sample 直接调用 C++
69         _Z21common_sampler_sample",
70     ("common_sampler_sample", "_Z21common_sampler_sample"):
71         "[SRC] llama.cpp sampler.cpp → common_sampler_sample 调用 C++
72         _Z21common_sampler_sample",
73     ("_Z21common_sampler_sample", "llama_sampler_sample"):
74         "[SRC] llama.cpp sampler.cpp → 双向: _Z21common_sampler_sample 也可回调
75         llama_sampler_sample",

```

```
72 # === llama.cpp 源码必然调用路径（批处理链） ===
73 # llama.cpp batch.h/c: llama_batch_add() 内部调用 common_batch_add()
74 ("llama_batch_add", "common_batch_add"):
75     "[SRC]_llama.cpp_batch.h_—_llama_batch_add_调用_common_batch_add",
76
77 # === llama.cpp 源码必然调用路径（调度器链） ===
78 # llama.cpp sched.h/c:
79 # ggml_backend_sched_graph_compute_async() 后必然调用同步
80 ("ggml_backend_sched_graph_compute_async", "ggml_backend_sched_synchronize"
    ):
```